Exception Handling Mechanism in Java

- It is a mechanism to handle the run time problems
- try, catch, throw, throws,finally

#Java has divided the run time problems into 2 categories

1. Error
   - The run time problem that does not belong to the code but is the problem of run time environment
   - HDD is crashed,
   - Java recommends not to handle the errors
   - Let the program crash
2. Exception
   - run time problems that occurs due to the logical errors in the program or problems that occur due to wrong inputs are all categorized as Exceptions
   - Java recommends to handle the exceptions

# Throwable class

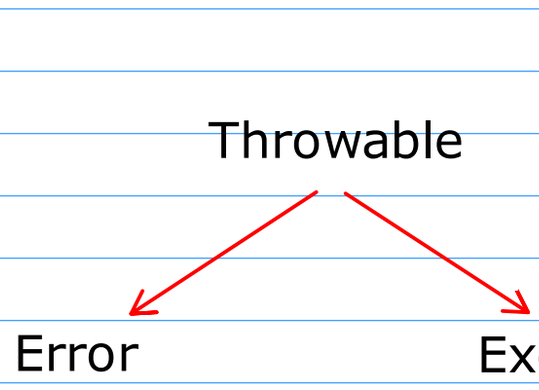
- it is a class declared in java which is the root class of all the errors and exceptions

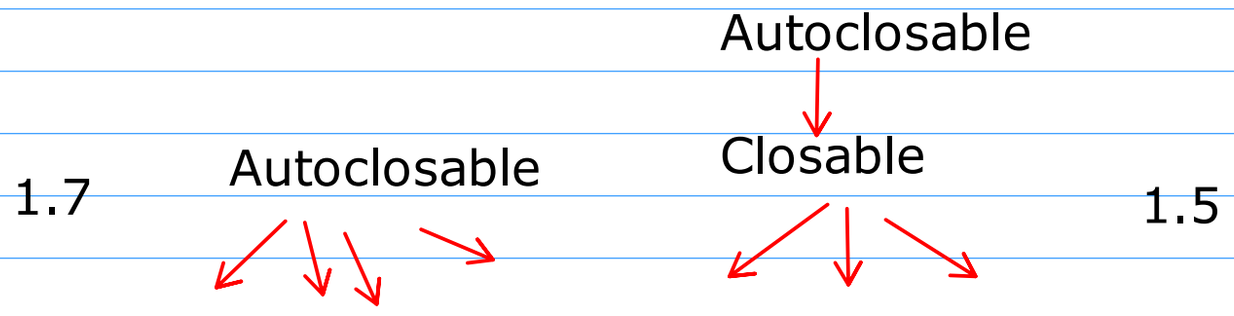
# Error class

- It is a subclass of Throwable

# Exception class

- It is a subclass of Throwable




- To handle the exception we have to use the below keywords

1. try
2. catch

```
try{

}catch(){
// handle the exception
}
```

```
try{

}finally{
 // close the resource
}
```

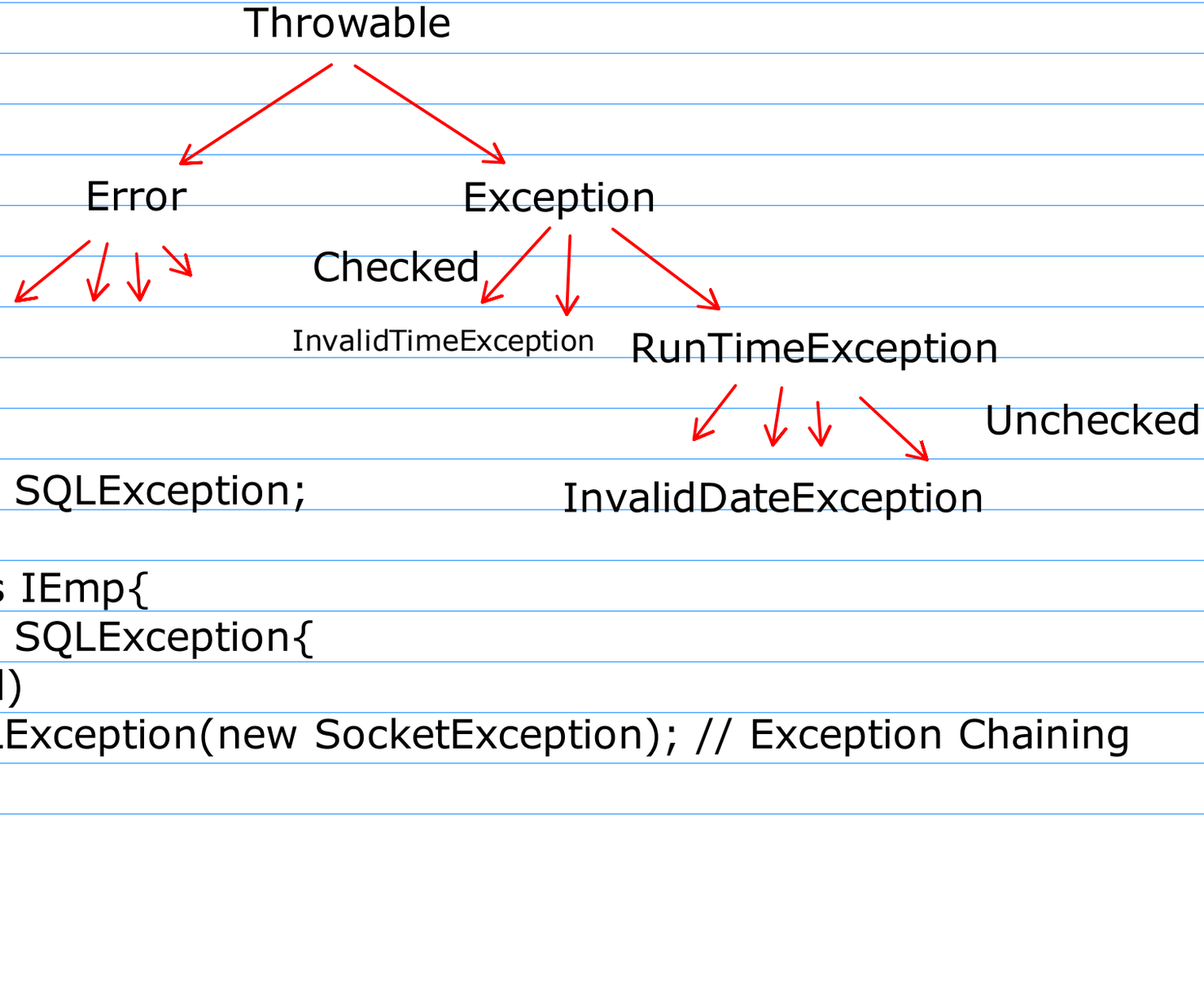
```
try(){

}
```

#Exception Categories
1. Checked Exception
    - These are the exceptions that are compulsary to be handled
    - we compulsary need to write the try-catch block to handle the exception, otherwise the program won't compile
    - Exception class itself and all its subclasses except RuntimeException class are all considered as Checked Exception
2. UnChecked Exception
    - These ar ethe exceptions that are optioanl to handle
    - It is optional to provide the try-catch, as theor wont be any compile time error.
    - The RuntimeException class itself and all its subclasses are considered as Unchecked Exceptions

throws
    - It is used to navigate the Checked Exception from current method towards the caller method

try
    - To identify/check  if their are any exceptions generated by the statements within it
catch
    - used to handle the exceptions
    - If any exception is generated from a try block then the matching catch block is used to handle it.
throw
    - It is used to generate new exceptions
throws
    - To navigate the exception generated in the method towards its caller method
finally
    - Whether their is exception or no any exception to close the resources we use finally block




```
interface IEmp{
void getEmployee() throws SQLException;
}
class Employee implements IEmp{
void getEmployee() throws SQLException{
        if(connection == null)
                throw new SQLException(new SocketException); // Exception Chaining
}
}
```